

中山大學

SUN YAT-SEN UNIVERSITY

实验 0x04 实验报告

用户程序与系统调用

姓　　名：　何亮

学　　号：　24325005

教学班号：　202520804（陈鹏飞老师班级）

专　　业：　计算机科学与技术

院　　系：　计算机学院

2025 学年第二学期

1. 思考题

1.1. 是否可以在内核线程中使用系统调用? 并借此来实现同样的进程退出能力? 分析并尝试回答。

系统调用指的是: 运行在用户空间的程序向操作系统内核请求需要更高权限运行的服务。系统调用是给特权级为 ring3 的代码来执行的。ring0 特权级的内核进程直接使用普通函数调用的方式来实现对应的功能。

因此, 我们不可以在内核线程内使用系统调用, 即使可以使用也可能会导致错误甚至是系统崩溃, 首先我们的系统调用只有一个栈, 如果我们在系统调用中使用系统调用, 会导致栈的混乱.

TODO.

1.2. 为什么需要克隆内核页表? 在系统调用的内核态下使用的是哪一张页表? 用户态程序尝试访问内核空间会被正确拦截吗? 尝试验证你的实现是否正确。

因为在用户进程中, 会需要用虚拟地址来访问内核的代码和数据(如发生中断或者系统调用等), 因此我们需要克隆内核的页表来保证上述虚拟地址可以正确地访问。

在系统调用的内核态下, 使用的是当前触发该系统调用的这个用户进程自己的页表。我们可以通过在系统调用的内核函数处理函数中输出用户进程的 `CR3` 寄存器值和当前 `CR3` 寄存器的值, 可以发现两者是一样的

```
[INFO] ysos_kernel::interrupt::syscall - current cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - process cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - current cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - process cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - current cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - process cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - current cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - process cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - current cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - process cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - current cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - process cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - current cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - process cr3: 0x164000
[INFO] ysos_kernel::interrupt::syscall - current cr3: 0x164000
```

尝试在 `hello` 中加入访问内核态起始地址的代码,

```rust
let kernel_ptr = 0xffffff0000000000u64 as *const i32;
let secret = unsafe { *kernel_ptr };

println!("The secret number is: {}", secret);
```

然后尝试运行，如下图，可以看到 `manager` 捕获到了这个非法访问，说明我们正确拦截了用户程序对内核空间的访问

```
[>] hello
Hello, world!!!
[WARN] ysos_kernel::proc::manager - Page fault at 0xffffff0000000000 wit
h protection violation.
[WARN] ysos_kernel::interrupt::exceptions - EXCEPTION: PAGE FAULT, ERROR
_CODE: PageFaultErrorCode(PROTECTION_VIOLATION | USER_MODE)

Trying to access: 0xffffff0000000000
Caused by PID: 3
InterruptStackFrame {
    instruction_pointer: VirtAddr(
        0x11110000101f,
    ),
    code_segment: SegmentSelector {
        index: 5,
        rpl: Ring3,
    },
    cpu_flags: RFlags(
```

1.3. 为什么在使用 `still_alive` 函数判断进程是否存活时，需要关闭中断？在不关闭中断的情况下，会有什么问题？

因为 `still_alive` 函数需要调用 `get_process_manager()` 来访问某个进程的状态，此时需要获得 `ProcessManager` 的自旋锁。如果我们不关闭中断，此时会由于时钟中断来临，时钟中断内部会进行进程的调度，也需要获得 `ProcessManager` 的锁，从而一直自旋等待，造成死锁；其次，如果时间恰好不会造成死锁，正确处理了进程调度，也可能会在其他中断事件发生的过程中修改了某些进程的状态，让一个进程的状态被 `kill` 掉，那么这个时候 `still_alive` 的返回值就可能会和实际不一样，造成错误。

1.4. 对于如下程序，使用 `gcc` 直接编译：

```c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

从本次实验及先前实验的所学内容出发，结合进程的创建、链接、执行、退出的生命周期，参考系统调用的调用过程（可以仅以 Linux 为例），解释程序的运行。

首先我们使用 `gcc` 编译，编译器通过预处理，编译，汇编，链接等将源代码与标准库代码等链接在一起，生成一个 ELF 文件。

然后在 Linux 中执行的时候，内核会通过 `fork()` 系统调用生成一个子进程，然后克隆内核的页表，分配 PID 等来创建这个进程。随后，加载并解析前面生成的 ELF 文件，建立页表映射和用户栈。最后从系统调用返回会用户态，进入程序入口点开始执行。

该程序使用了 `printf` 函数，这个函数的参数会现在用户态进行初步的处理，然后触发 `write` 系统调用，用 `rdi` 等寄存器传参，随后进入 Ring0 特权级的内核态开始处理串口输出。

在输出了 `Hello World!` 后，系统调用返回 Ring3 特权级的用户态，最后 `return 0` 会调用 `exit` 系统调用（过程和上述类似，这里不再赘述，这里传入的参数为 `0`）。然后，该进程变为僵尸进程，内核通过调用 `wait` 来回收资源。

1.5. `x86_64::instructions::hlt` 做了什么？为什么这样使用？为什么不可以在用户态中的 `wait_pid` 实现中使用？

`x86_64::instructions::hlt` 用于把当前指令执行的过程挂起，让出 CPU，直到下一次的外部中断来临，然后继续执行下面的代码。我们在 `wait()` 函数中，我们需要的是等待线程结束然后返回。如果我们一直使用循环来进行忙等待，就会一直占用 CPU，导致资源浪费，使用这个函数可以让出 CPU 供其他线程使用，提升 CPU 利用率，然后在时钟中断来临的时候可以继续检查线程是否 `Dead`，保证原功能正常执行的同时提升性能。

由于 `x86_64::instructions::hlt` 是一条 Ring0 特权级指令，只能在内核态下执行，因此不可以在用户态 Ring3 特权级下的 `wait_pid` 中使用。

1.6. 只有当进行用户输入（触发了串口输入中断）的时候，会触发奇怪的 Page Fault，然而进程切换、内存分配甚至 fork 等系统调用都很正常。请查阅资料，了解特权级栈的作用，实验说明这一系列中断的触发过程，尝试解释这个现象。

- 可以使用 intdbg 参数，或 ysos.py -i 进行数据捕获。
- 留意 0x0e 缺页异常和缺页之前的中断的信息。
- 注意到一个不应当存在的地址……？